Lab — Aula 1: Fundamentos de Orquestração com Airflow

> Pipeline ETL completo: Open-Meteo API → Pandas → PostgreSQL

Estrutura do projeto

```
lab-airflow/
├── docker-compose.yml          ← ambiente completo (Airflow + 2 bancos)
├── dags/
│   ├── clima_etl.py            ← DAG principal do lab
│   └── clima_etl_avancado.py   ← extensão (fan-out paralelo + branch)
├── scripts/
│   ├── testar_dag_local.py     ← testa as funções sem o Airflow
│   └── inspecionar_resultado.py← consulta o banco após a execução
├── sql/
│   └── init.sql                ← cria as tabelas no postgres-lab
├── logs/                       ← criada automaticamente pelo Airflow
└── plugins/                    ← (vazia por enquanto)
```

Pré-requisitos

Ferramenta	Versão mínima	Verificar com
Docker Desktop	4.x	`docker --version`
Docker Compose	v2 (embutido)	`docker compose version`
Python	3.9+ (opcional, para scripts locais)	`python --version`

> **RAM mínima:** 4 GB livres para o Docker. O Airflow com LocalExecutor usa ~1.5 GB.

Passo 1 — Subir o ambiente

```
# Clone o repositório (ou descompacte o zip do lab)
cd lab-airflow

# Criar pastas necessárias
mkdir -p logs plugins

# SOMENTE LINUX: permitir que o container grave nos logs
chmod 777 logs

# Subir todos os serviços em background
docker compose up -d

# Acompanhar a inicialização (aguardar "healthy" em todos)
docker compose ps

# Ver logs em tempo real (opcional)
docker compose logs -f airflow-webserver
```

Aguarde até ver `healthy` nas colunas de status:

```
NAME                  STATUS
airflow-meta-db       healthy
airflow-lab-db        healthy
airflow-scheduler     healthy
airflow-webserver     healthy
```

⏱️ Primeira vez: pode levar 2-3 minutos para baixar as imagens.

Passo 2 — Acessar a UI

Abra no navegador: **http://localhost:8080**

Campo	Valor
Usuário	`admin`
Senha	`admin`

Passo 3 — Ativar e executar o DAG

1. Na lista de DAGs, localize **`clima_etl`**
2. Clique no toggle à esquerda para **ativar** (passa de cinza para azul)

3. Clique no botão ▶ **Trigger DAG** (ícone de play) → confirme
4. Observe as tasks ficando **verde** uma a uma no Grid View

> 💡 A primeira execução pode levar 1-2 min enquanto o Airflow instala as dependências Python no worker.

Passo 4 — Explorar a UI

Logs de uma task

- Clique em qualquer task colorida no Grid View
- Selecione **Log** no menu lateral

Gantt Chart

- Na página do DAG, clique em uma run específica
- Selecione a aba **Gantt**
- Observe o tempo de cada task — qual foi a mais lenta?

XComs

- Menu superior: **Admin → XComs**
- Filtre por `dag_id = clima_etl`
- Veja os dados trocados entre as tasks

Connection criada automaticamente

- Menu superior: **Admin → Connections**
- Localize `postgres_lab` — criada pelo `airflow-init`

Passo 5 — Inspecionar os dados no banco

O banco de destino está acessível em `localhost:5433`:

```
# Usando psql (se instalado)
psql -h localhost -p 5433 -U lab -d labdb

# Ou usando o container diretamente
docker exec -it airflow-lab-db psql -U lab -d labdb

# Consultas úteis:
SELECT cidade, COUNT(*), ROUND(AVG(temperatura_c)::numeric, 1) as media_c
FROM clima
GROUP BY cidade;

SELECT * FROM v_clima_resumo;
```

Ou usando o script Python (requer `pip install psycopg2-binary pandas`):

```
python scripts/inspecionar_resultado.py
python scripts/inspecionar_resultado.py --data 2024-03-15
```

Passo 6 — Simular uma falha e observar retry

1. Edite `dags/clima_etl.py`
2. Na função `buscar_clima()`, altere a URL para algo inválido:

   ```
   url = "https://api.open-meteo.com/ROTA_INVALIDA"
   ```

3. Salve o arquivo — o Airflow detecta automaticamente em ~30s
4. Acione o DAG novamente
5. Observe no Grid View:
   - Task ficando **laranja** (retry em andamento)
   - Logs mostrando as tentativas com backoff exponencial
   - Task ficando **vermelha** após esgotar os retries
6. Corrija a URL, salve, e reexecute — as tasks ficam verdes

Extensão — DAG avançado (para quem terminar cedo)

O arquivo `dags/clima_etl_avancado.py` demonstra:

- **Fan-out paralelo**: cada cidade processada em task_group independente
- **BranchPythonOperator**: alerta condicional se temperatura > 35°C
- **on_failure_callback**: log de alerta em caso de falha
- **SLA**: limite de 10 minutos por task

Para ativá-lo, repita os passos 3-5 com o DAG `clima_etl_avancado`.

Questões para reflexão:

- No Gantt Chart, as 3 cidades rodam em paralelo ou sequencialmente?
- O que muda no Graph View comparado ao DAG simples?
- Como o BranchPythonOperator aparece no Grid View?

Testar as funções localmente (sem Docker)

```
pip install requests pandas

# Testa fetch + transform e salva resultados em /tmp/
python scripts/testar_dag_local.py

# Testa só o fetch
python scripts/testar_dag_local.py --step fetch

# Testa só o transform (usa dados do fetch anterior)
python scripts/testar_dag_local.py --step transform
```

Derrubar o ambiente

```
# Para os containers (mantém os dados)
docker compose down

# Para e apaga tudo (volumes incluídos)
docker compose down -v
```

Entregável

Ao final do lab, compartilhe no chat/e-mail:

- [ ] Print do **Gantt Chart** com todas as tasks verdes
- [ ] Print da tela de **XComs** mostrando os dados trocados
- [ ] (Extensão) Print do DAG avançado com as tasks paralelas no Gantt

---

Troubleshooting

Problema	Solução
`localhost:8080` não abre	`docker compose ps` — verificar se `airflow-webserver` está `healthy`
DAG não aparece na UI	Verificar erros com `docker compose logs airflow-scheduler`
Task travada em "running"	Clicar na task → Clear → Confirm
Erro de Connection	Admin → Connections → verificar `postgres_lab`
`PermissionError` em `logs/` (Linux)	`mkdir -p logs plugins && chmod 777 logs` e `docker compose restart`
Sem espaço em disco	`docker system prune` para liberar imagens antigas
Erro de memória	Aumentar RAM disponível no Docker Desktop (Settings → Resources)